

INTRODUCCIÓN A JAVASCRIPT

Conceptos fundamentales, sintaxis y buenas prácticas

Apuntes revisados y ampliados

Documento elaborado a partir del fastbook original de Robert Da Corte

Contenido

1. Introducción a JavaScript	3
1.1. Breve historia y evolución del lenguaje	3
1.2. ¿Qué es JavaScript?	3
2. Sentencias y expresiones	4
3. Variables y tipos de datos	4
3.1. Declaración de variables (var, let y const)	4
3.2. Tipos de datos	5
3.3. Conversión de tipos y coerción	5
4. Operadores	6
4.1. Operadores aritméticos	6
4.2. Operadores de comparación	6
4.3. Operadores lógicos	7
4.4. Operadores de asignación	7
5. Estructuras de control	7
5.1. Condicionales	7
5.2. Bucles	8
6. Funciones	9
6.1. Declaración y llamada de funciones	9
6.2. Parámetros y argumentos	9
6.3. Funciones de flecha (arrow functions)	9
6.4. Alcance de las variables (scope) y hoisting	10
7. Objetos	10
8. Arrays	11
9. Document Object Model (DOM)	11
10. Eventos	12
10.1. Bubbling	12
11. Qué hemos aprendido	13

1. Introducción a JavaScript

JavaScript es un lenguaje de programación de alto nivel, interpretado y multiparadigma, creado originalmente para dotar de dinamismo a las páginas web. Hoy en día es una pieza esencial del desarrollo web moderno, tanto en el navegador como en el servidor.

1.1. Breve historia y evolución del lenguaje

JavaScript fue creado por **Brendan Eich** en Netscape Communications, a mediados de los años 90. En septiembre de 1995 se lanzó como parte de Netscape Navigator 2.0 bajo el nombre de **LiveScript**. Poco después, aprovechando la popularidad de Java, Netscape lo renombró a **JavaScript**, aunque ambos lenguajes no tienen relación técnica directa.

Para que otros navegadores pudieran implementarlo de forma coherente, Netscape presentó el lenguaje ante **Ecma International** en 1996. De ahí surgió el estándar **ECMA-262** (junio de 1997), que define la especificación conocida como **ECMAScript**. Desde entonces, JavaScript es simplemente una implementación de ese estándar.

Microsoft, por su parte, desarrolló JScript, una versión prácticamente idéntica pero con matices legales que provocó incompatibilidades entre navegadores durante la llamada guerra de navegadores. En diciembre de 1999 llegó ECMAScript 3 (ES3), edición que se mantuvo vigente durante toda la década de los 2000.

El gran salto llegó en 2015 con **ECMAScript 2015, también conocido como ES6**, que introdujo `let` y `const`, las arrow functions, las clases, el destructuring y otras muchas mejoras. Es la versión sobre la que se trabaja hoy en el mundo empresarial y la que se usa como referencia en este documento.

★ Lo más importante

- ▶ JavaScript nació en 1995 de la mano de Brendan Eich (Netscape) bajo el nombre de LiveScript.
- ▶ El estándar oficial del lenguaje se llama ECMAScript (ECMA-262); JavaScript es una implementación de dicho estándar.
- ▶ ES6 (2015) es la versión de referencia actual: introdujo `let`, `const`, arrow functions, clases y destructuring.

1.2. ¿Qué es JavaScript?

JavaScript es un lenguaje de programación de **alto nivel, interpretado y multiparadigma**. Cada uno de estos términos describe una característica distinta del lenguaje.

Lenguaje de alto nivel — Internamente, el ordenador solo entiende electricidad encendida o apagada (1 y 0), lo que se conoce como **sistema binario** o **lenguaje máquina**. Un lenguaje de alto nivel tiene una gramática pensada para que las personas la entiendan con facilidad; después, un proceso de traducción la convierte a lenguaje máquina.

Lenguaje interpretado — La traducción a lenguaje máquina se realiza en tiempo real mediante un **intérprete**, integrado en los navegadores (Chrome, Firefox, Safari, Edge...). Esto lo diferencia de los lenguajes **compilados**, que generan de antemano un archivo ya traducido (por ejemplo, un .exe).

Lenguaje multiparadigma — Soporta múltiples estilos de programación: distintas formas de generar y organizar el código según las necesidades del proyecto.

★ Lo más importante

- ▶ Alto nivel: sintaxis legible por personas, que luego se traduce a lenguaje máquina.
- ▶ Interpretado: el navegador traduce el código en tiempo real, sin generar un archivo compilado aparte.
- ▶ Multiparadigma: admite distintos estilos y formas de organizar el código.

2. Sentencias y expresiones

Sentencia — Es una instrucción que se le da al motor de JavaScript para que la ejecute. Las sentencias son los bloques de construcción de cualquier programa y determinan su flujo y su lógica.

Por convención, las sentencias terminan con punto y coma (;). Aunque JavaScript dispone de la **inserción automática de punto y coma** (ASI) y en muchos casos no es obligatorio, se recomienda incluirlo siempre para evitar errores inesperados.

```
// Sentencias
let x = 5;      // Sentencia
console.log(x); // Sentencia
```

Expresión — Es cualquier fragmento de código que evalúa un valor. En JavaScript, casi cualquier trozo de código que produce un valor puede considerarse una expresión. Una expresión puede formar parte de una sentencia: por ejemplo, en **let z = x + y;**, el fragmento **x + y** es la expresión.

★ Lo más importante

- ▶ Sentencia = instrucción completa que forma parte del programa.
- ▶ Expresión = fragmento de código que se evalúa y devuelve un valor; puede vivir dentro de una sentencia.
- ▶ Buena práctica: termina siempre tus sentencias con punto y coma (;).

3. Variables y tipos de datos

Una variable puede imaginarse como un *vaso* que utilizamos para contener distintas cosas: agua, zumo, monedas... En JavaScript, una variable almacena datos que podemos usar en nuestro código: números, cadenas de texto, booleanos, objetos, etc.

```
// vamos a definir una variable age
let age = 30;
let message = 'Tienes ' + age + ' años.';
console.log(message); // Tienes 30 años.
```

3.1. Declaración de variables (var, let y const)

JavaScript dispone de tres palabras reservadas para declarar una variable:

- **var** — Era la única forma de declarar variables antes de ES6. Su uso está en desuso porque introduce efectos secundarios (hoisting a nivel de función, ausencia de block scope).
- **let** — Introducida en ES6, sustituye a var. Permite cambiar el valor a lo largo de la ejecución del programa y resuelve los problemas de block scope y hoisting.
- **const** — Se comporta como let, pero se utiliza para declarar constantes: variables que no deberían cambiar de valor durante la vida del programa.

Una misma variable declarada con let o var puede contener distintos tipos de datos a lo largo de la vida del programa. Esta característica sitúa a JavaScript en la categoría de lenguajes de **tipado dinámico**.

```
let age = 30;
// asignamos ahora un string
age = 'treinta';
let message = 'Tienes ' + age + ' años.';
console.log(message); // Tienes treinta años.
```

3.2. Tipos de datos

En JavaScript existen seis tipos de datos primitivos:

- **number** — Números enteros o de punto flotante; sirve para cálculos matemáticos.
- **symbol** — Introducido en ES6; crea identificadores únicos e inmutables, útil para propiedades "privadas" de un objeto.
- **string** — Representa texto: una secuencia de caracteres (letras, números, símbolos, espacios).
- **boolean** — Representa valores de verdad: true o false. Se usa sobre todo para validar condiciones.
- **BigInt** — Introducido en ECMAScript 2020, representa números enteros arbitrariamente grandes (más allá del límite de number).
- **undefined** — Indica la ausencia de un valor: una variable sin asignar o una propiedad no definida.

Existe además un tipo especial llamado **null**, que representa la ausencia *intencionada* de un valor (a diferencia de undefined, que indica que la variable ni siquiera se ha inicializado). Más adelante veremos también las estructuras de datos complejas: **objetos** y **arrays**.

3.3. Conversión de tipos y coerción

La **conversión de tipos** ocurre cuando un dato pasa de un tipo a otro, de forma implícita o explícita. La **coerción** es la conversión automática que realiza JavaScript en determinadas situaciones.

```
// Coerción
let ageString = '25';
let ageInt = Number(ageString); // 25 (número)

let num = 42;
let str = String(num); // '42' (cadena)

let isValid = true;
let numIsValid = Number(isValid); // 1 (true equivale a 1)

let stringValue = 'Hola';
let boolValue = Boolean(stringValue); // true (cadena no vacía)
```

★ Lo más importante

- ▶ Prefiere `let` y `const` frente a `var`; usa `const` siempre que el valor no deba cambiar.
- ▶ Existen 6 tipos primitivos: `number`, `symbol`, `string`, `boolean`, `BigInt` y `undefined` (más `null`, un caso especial).
- ▶ JavaScript es de tipado dinámico: una misma variable puede cambiar de tipo a lo largo del programa.
- ▶ Vigila la coerción automática de tipos: puede producir resultados inesperados si no se controla.

4. Operadores

Los operadores son símbolos especiales que permiten realizar cálculos, comparaciones y manipulaciones sobre variables y valores.

4.1. Operadores aritméticos

- **Suma** `+` — Suma dos números o concatena cadenas de texto.
- **Resta** `-` — Resta un número de otro.
- **Multiplicación** `*` — Multiplica dos números.
- **División** `/` — Divide un número por otro.
- **Módulo** `%` — Devuelve el resto de la división entre dos números.

```
const posX = 10;
const posY = 5;
let add = posX + posY;      // 15
let subtract = posX - posY; // 5
let multiply = posY * posY; // 25
let divide = posX / posY;   // 2
let remainder = posX % posY; // 0

// El operador de suma también concatena strings
console.log('¡' + 'Hola' + ' ' + 'Mundo' + '!'); // ¡Hola Mundo!
```

4.2. Operadores de comparación

Comparan dos valores y devuelven un booleano (`true` o `false`):

- Igual `==` — compara si dos valores son iguales (con coerción de tipos).
- Diferente `!=` — compara si dos valores no son iguales.
- **Estrictamente igual** `===` — compara si dos valores son iguales y del mismo tipo.
- **Estrictamente no igual** `!==` — compara si dos valores no son iguales o no son del mismo tipo.
- Mayor que `>` / Menor que `<` — comparan la magnitud de dos valores.
- Mayor o igual que `>=` / Menor o igual que `<=` — comparan incluyendo la igualdad.

```
const a = 10; // number
const b = '10'; // string

console.log(a == b); // true  -> compara solo el valor
console.log(a === b); // false -> compara valor y tipo
```

4.3. Operadores lógicos

- **AND lógico &&** — devuelve true si ambos operandos son true.
- **OR lógico ||** — devuelve true si al menos uno de los operandos es true.
- **NOT lógico !** — niega el valor de un operando (true pasa a false y viceversa).

4.4. Operadores de asignación

- Asignación = — asigna el valor de la derecha a la variable de la izquierda.
- Adición +=, resta -=, multiplicación *= y división /= — operan sobre el valor actual de la variable y reasignan el resultado.

```
let counter = 5;
counter += 3; // equivalente a: counter = counter + 3;
```

★ Lo más importante

- ▶ Usa siempre === y !== (comparación estricta) para evitar sorpresas por coerción de tipos.
- ▶ El operador + también sirve para concatenar cadenas de texto, no solo para sumar números.
- ▶ &&, || y ! permiten combinar y negar condiciones booleanas.

5. Estructuras de control

Las estructuras de control son bloques de código que permiten tomar decisiones y controlar el flujo de ejecución de un programa: ejecutar acciones condicionales y repetir tareas.

5.1. Condicionales

if-else — Ejecuta un bloque de código si se cumple una condición y otro distinto si no se cumple. Puede evaluar múltiples condiciones con else if.

```
const hour = 15;

if (hour < 12) {
  console.log('Buenos días.');
```

```
} else if (hour < 18) {
  console.log('Buenas tardes.');
```

```
} else {
  console.log('Buenas noches.');
```

```
}
```

switch — Evalúa una variable frente a distintos casos y ejecuta el bloque correspondiente al que coincide. Usa tres palabras reservadas: **case** para cada valor posible, **break** (obligatorio) para salir del switch, y **default** para el caso no contemplado.

```
let dayOfTheWeek = 'Martes';

switch (dayOfTheWeek) {
  case 'Lunes':
    console.log('Es el primer día de la semana.');
```

```

    break;
  case 'Martes':
    console.log('Es el segundo día de la semana.');
```

```
    break;
```

```
  default:
```

```
    console.log('Es fin de semana.');
```

```
}
```

★ Regla clave

- ▶ No olvides el break en cada case: si lo omites, la ejecución continúa en el siguiente caso (fallthrough).

Operador ternario — Permite escribir una evaluación condicional en una sola línea. Su sintaxis es: **condición ? resultadoSiCumple : resultadoSiNoCumple**.

```

const amount = 20;
const coffeePrice = 10;

const message = amount > coffeePrice
  ? 'Fondos suficientes para la compra'
  : 'No dispone de fondos suficientes';
```

5.2. Bucles

Permiten repetir un bloque de instrucciones mientras se cumpla una condición. Existen tres tipos principales:

- **while** — comprueba la condición antes de cada iteración; si es verdadera, continúa ejecutándose.
- **for** — ideal cuando se conoce el número de iteraciones; se compone de inicialización, condición y actualización.
- **do-while** — ejecuta el bloque al menos una vez antes de comprobar la condición.

```

// for
for (let i = 1; i <= 5; i++) {
  console.log('Número: ' + i);
}

// do-while
let count = 1;
do {
  console.log('Número: ' + count);
  count++;
} while (count <= 5);
```

★ Lo más importante

- ▶ if-else y switch sirven para tomar decisiones; while, for y do-while sirven para repetir tareas.
- ▶ El operador ternario es útil para condicionales cortos de una sola línea.

- ▶ do-while garantiza que el bloque se ejecute al menos una vez, aunque la condición sea falsa desde el principio.

6. Funciones

Las funciones son bloques de código reutilizables que pueden recibir datos de entrada (argumentos), realizar operaciones y devolver un resultado. Son esenciales para organizar y modularizar el código.

6.1. Declaración y llamada de funciones

Una función se declara con la palabra reservada **function**, seguida de un nombre, unos paréntesis (que pueden contener parámetros) y un bloque de código.

```
// Declaración de una función
function sayHi(name) {
  console.log('¡Hola, ' + name + '!');
}

// Llamada a la función
sayHi('Juan'); // Resultado: ¡Hola, Juan!
```

6.2. Parámetros y argumentos

Parámetros — Variables que reciben valores dentro de la función. **Argumentos** — Los valores reales que se pasan a la función cuando se la llama.

```
function add(a, b) { // a y b son parámetros
  return a + b;
}

const result = add(3, 5); // 3 y 5 son argumentos -> result = 8
```

Las funciones también pueden ser **anónimas** (sin nombre propio) y asignarse a una variable. La palabra reservada **return** hace que la función devuelva un valor; su uso es opcional, según la lógica de la función.

6.3. Funciones de flecha (arrow functions)

Introducidas en ES6, ofrecen una sintaxis más concisa para escribir funciones.

```
// Sin parámetros
const showHelloWorld = () => console.log('Hola Mundo');

// Un solo parámetro (los paréntesis son opcionales)
const showAddInConsole = result => console.log(result);

// Dos o más parámetros: paréntesis obligatorios
const add = (x, y) => x + y;

// Con más de una sentencia, necesita cuerpo y return explícito
const addAndMultiplyBy = (value, multiplier) => {
  const result = value + multiplier;
  return result * multiplier;
};
```

6.4. Alcance de las variables (scope) y hoisting

El **alcance** (scope) de una variable determina dónde es accesible dentro del código. Las variables declaradas dentro de una función solo son visibles dentro de ella: es lo que se llama **alcance local**.

El **hoisting** es una particularidad del intérprete de JavaScript: antes de ejecutar el código, "sube" las declaraciones de variables y funciones al inicio de su contexto (global o de bloque), y solo después las inicializa con su valor.

- **var** — sufre hoisting a nivel de función; si se lee antes de inicializarse, su valor es undefined (no da error).
- **let / const** — tienen alcance de bloque y no sufren la elevación del mismo modo: acceder a ellas antes de su declaración lanza un ReferenceError. A ese intervalo se le llama temporal dead zone.

★ Lo más importante

- Una función puede llevar o no return, según si necesita devolver un valor.
- Las arrow functions son la forma moderna y más concisa de escribir funciones en ES6.
- var sufre hoisting y se inicializa como undefined; let/const entran en la temporal dead zone y lanzan error si se usan antes de declararse.

7. Objetos

Los objetos son estructuras fundamentales en JavaScript que permiten agrupar datos y funcionalidades relacionadas en una única entidad: propiedades (datos) y métodos (funciones).

Se crean habitualmente con la sintaxis de llaves { }, asignando después propiedades y métodos.

```
const person = {
  name: 'Juan',
  age: 30,
  sayHi: function() {
    console.log('¡Hola! Mi nombre es ' + this.name + ' y tengo ' + this.age + ' años.');
```

```
  }
};

console.log(person.name); // Juan
person.sayHi();           // ¡Hola! Mi nombre es Juan y tengo 30 años.
```

Las propiedades pueden accederse mediante notación de punto (**objeto.propiedad**) o notación de corchetes (**objeto["propiedad"]**). Los métodos se acceden y llaman de la misma manera.

★ Lo más importante

- Propiedades = datos del objeto; métodos = funciones que representan sus acciones.

- ▶ Dos formas de acceder: notación de punto (obj.propiedad) y notación de corchetes (obj["propiedad"]).

8. Arrays

Los arrays (o arreglos) son estructuras de datos que permiten almacenar y organizar múltiples elementos relacionados en una sola variable. Cada elemento tiene un **índice**, empezando siempre en **0**.

```
const fruits = ['manzana', 'pera', 'platano', 'fresa'];

console.log(fruits[0]); // manzana
console.log(fruits[1]); // pera
```

Los arrays disponen de numerosos métodos incorporados para agregar, eliminar y modificar elementos. Los más usados son:

- **push()** / **pop()** — añaden / eliminan un elemento al final del array.
- **unshift()** / **shift()** — añaden / eliminan un elemento al inicio del array.
- **indexOf()** — busca un elemento y devuelve su índice.
- **slice(inicio, fin)** — devuelve un nuevo array con los elementos seleccionados, sin modificar el original.
- **splice(inicio, cantidad)** — elimina o reemplaza elementos, modificando el array original.
- **concat()** — une dos o más arrays en uno nuevo.

```
const fruits = ['manzana', 'pera', 'platano', 'fresa'];

fruits.push('naranja'); // ['manzana', 'pera', 'platano', 'fresa', 'naranja']
fruits.shift();          // ['pera', 'platano', 'fresa', 'naranja']
const position = fruits.indexOf('platano'); // 1
```

★ Lo más importante

- ▶ El primer índice de un array siempre es 0.
- ▶ **slice()** no modifica el array original; **splice()** sí lo modifica.
- ▶ Existen muchos más métodos de array, de uso más avanzado, que se irán viendo con la práctica.

9. Document Object Model (DOM)

El DOM es una **representación estructurada y programática** de un documento web: convierte todo el contenido de una página en objetos que pueden manipularse, en su mayoría, mediante JavaScript. Se representa como un **árbol de nodos** (elementos HTML, atributos, texto...).

Una vez cargada la página, es posible modificar elementos, atributos y estilos; añadir o eliminar nodos; y responder a eventos, todo en tiempo real y sin necesidad de recargar la página.

JavaScript dispone de varios objetos para manipular el DOM:

- **document** — Representa el propio documento. Métodos habituales: `getElementById(id)`, `querySelector(selector)`, `querySelectorAll(selector)`, `createElement(tagName)`, `createTextNode(data)`.
- **element** — Representa un elemento concreto del DOM. Propiedades y métodos habituales: `innerHTML`, `textContent`, `setAttribute()`, `getAttribute()`, `classList`, `appendChild()`, `removeChild()`, `addEventListener()`.
- **window** — Representa la ventana del navegador. Incluye `alert()`, `prompt()`, `innerWidth`, `innerHeight`, entre otros.

```
const element = document.querySelector('p.title');
element.textContent = 'Nuevo texto';
element.classList.add('activo');
```

Nota

- ▶ Aunque el DOM se manipula sobre todo con JavaScript, el DOM en sí es independiente de cualquier lenguaje de programación: está diseñado para ser accesible desde diversos lenguajes.

★ Lo más importante

- ▶ El DOM representa la página como un árbol de nodos.
- ▶ `document` es la puerta de entrada al contenido; `element` permite manipular un nodo concreto; `window` representa el entorno del navegador.

10. Eventos

Los eventos son acciones o sucesos que ocurren en el navegador (o, en Node.js, en el sistema) a los que podemos responder usando JavaScript: un clic, una tecla pulsada, el envío de un formulario, una solicitud HTTP recibida, etc.

JavaScript permite *escuchar* estos eventos y *reaccionar* a ellos mediante `addEventListener()`, que recibe el nombre del evento (como cadena de texto) y la función que se ejecutará cuando ocurra.

```
// Suponemos que existe un elemento <button> en el HTML
const button = document.querySelector('button');

button.addEventListener('click', function() {
  alert('¡Haz clicado un botón!');
});
```

10.1. Bubbling

Cuando ocurre un evento en un elemento, no solo afecta a ese elemento sino también a todos sus ancestros en el DOM: el evento "burbujea" hacia arriba. Si un botón y su div contenedor tienen ambos un listener para 'click', **se ejecutarán los dos**, primero el del botón y después el del div.

```
document.getElementById('container').addEventListener('click', () => {
  console.log('Div contenedor fue clicado!');
```

```
});  
  
document.querySelector('#button').addEventListener('click', () => {  
  console.log('Botón hijo fue clicado!');  
});  
// Resultado al pulsar el botón:  
// 'Botón hijo fue clicado!'  
// 'Div contenedor fue clicado!'
```

Si no se desea esta propagación, puede detenerse con `event.stopPropagation()`, dentro del manejador del evento.

★ Lo más importante

- ▶ `addEventListener` recibe el nombre del evento como texto y una función que se ejecuta al producirse.
- ▶ Por defecto, los eventos hacen bubbling: burbujan desde el elemento de origen hacia sus ancestros.
- ▶ `event.stopPropagation()` detiene esa propagación cuando no es deseada.

11. Qué hemos aprendido

A lo largo de este documento hemos recorrido los conceptos fundamentales de JavaScript, desde su origen como lenguaje de scripting en el navegador hasta su papel central en el desarrollo web actual.

- **Variables y tipos de datos:** cómo declarar variables, su alcance, y cómo almacenar números, cadenas, booleanos y otros tipos primitivos.
- **Operadores:** aritméticos, de comparación, lógicos y de asignación, para realizar cálculos y construir condiciones.
- **Estructuras condicionales y bucles:** if-else, switch, el operador ternario, y los bucles while, for y do-while.
- **Funciones:** declaración, parámetros, argumentos, arrow functions, scope y hoisting.
- **Objetos y arrays:** cómo agrupar datos y comportamiento, y cómo almacenar y manipular colecciones de elementos.
- **DOM y eventos:** cómo representa el navegador una página web y cómo reaccionar a las acciones del usuario.

Estos conceptos forman los cimientos de la programación en JavaScript y son esenciales para crear aplicaciones interactivas y funcionales, tanto en la web como en otros entornos.

¡Enhorabuena! Resumen completado

- ▶ La clave del buen desarrollador es seguir explorando y practicando: solo así se pueden construir aplicaciones más complejas y resolver desafíos cada vez mayores.